

Day 3: Database Scaling

1. What is Database Scaling?

Database scaling means increasing the capacity of a database so that it can handle more users, more data, and more requests without becoming slow or unavailable.

For example, suppose an e-commerce application initially has 1,000 users. A single database server may easily handle all the requests. If the application grows to 10 million users, the same server may become overloaded. We then need to scale the database.

Why is Database Scaling Needed?

As an application grows, the number of users, amount of data, and number of database requests increase.

Without scaling, we may face:

- Slow queries
- High response time
- Database overload
- Application crashes
- Storage limitations
- Poor user experience

The main purpose of database scaling is to maintain good performance and availability as the application grows.

2. Vertical Scaling

Vertical scaling, also called scale-up, means making the existing database server more powerful.

For example, suppose a database server currently has 8 GB RAM, a 4-core CPU and 500 GB storage.

We can upgrade the same server to 64 GB RAM, a 16-core CPU and 2 TB SSD storage.

In vertical scaling, we can increase:

- RAM
- CPU
- Storage capacity
- Disk speed
- Network capacity

A simple real-world example is a restaurant. If the restaurant becomes crowded, instead of opening another restaurant, we make the same restaurant bigger and add more resources.

Vertical scaling is relatively simple because we are still working with one main server.

However, it has a limitation. We cannot keep increasing the hardware forever. Powerful machines also become increasingly expensive.

Easy way to remember:

Vertical scaling means making one machine bigger and more powerful.

3. Horizontal Scaling

Horizontal scaling, also called scale-out, means adding more database servers instead of continuously increasing the power of one server.

For example, instead of using one database server, we might use five database servers and distribute the work among them.

A simple restaurant example is opening more branches instead of making one restaurant extremely large.

Horizontal scaling is commonly used by large applications because they may have millions of users and enormous amounts of data.

Its major advantage is that we can continue adding machines as the system grows.

However, horizontal scaling makes the system more complicated. We need to decide how data should be distributed, how servers communicate, how failures are handled, and how consistency is maintained.

Easy way to remember:

Vertical scaling means a bigger machine.

Horizontal scaling means more machines.

4. Database Replication

Database replication means maintaining copies of the same data on multiple database servers.

For example, suppose we have one primary database and two replicas.

When new data is added to the primary database, that change is also copied to the replicas.

Therefore, the primary and replicas generally contain the same information.

Replication is useful for improving availability and read performance.

If thousands of users are reading data, instead of sending every request to the primary database, some read requests can be handled by replicas.

Replication also provides redundancy. If one copy of the database is unavailable, other copies may still exist.

One important problem with replication is replication lag. Sometimes a change made on the primary takes a small amount of time to reach a replica. During that period, the replica may return slightly older data.

Easy way to remember:

Replication means same data, multiple copies.

5. Primary-Replica or Master-Slave Architecture

Primary-Replica is a common database replication architecture. It was traditionally called Master-Slave architecture.

The primary database is generally responsible for write operations such as inserting, updating and deleting data.

The replicas receive copies of changes made on the primary and are commonly used for read operations.

For example, when a user places an order, the order may first be written to the primary database. The change is then replicated to other database servers.

When users simply want to view their previous orders, those read requests can be sent to replicas.

This reduces the load on the primary database.

If the primary fails, depending on the system design, one replica may be promoted to become the new primary. This process is known as failover.

6. Database Partitioning

Database partitioning means dividing a very large table or dataset into smaller logical parts.

For example, suppose an Orders table contains ten crore records.

Instead of keeping all orders together, we can create separate partitions for orders from 2024, 2025 and 2026.

If we want only the orders from 2026, the database can focus on the relevant partition rather than unnecessarily processing older partitions.

Partitioning can therefore improve performance and make large amounts of data easier to manage.

Data can be partitioned using different criteria such as:

- Date
- Region
- User ID
- Category
- Functionality

Partitioning by functionality means separating data according to different functions of the application.

For example, user information, product information, order information and payment information can be logically separated according to their purpose.

Easy way to remember:

Partitioning means breaking large data into smaller manageable pieces.

7. Sharding

Sharding means dividing a large amount of data and storing different portions of that data on different database servers.

Suppose an application has ten crore users.

The first group of users might be stored on Server 1, the second group on Server 2, and the third group on Server 3.

Each server stores a different portion of the complete dataset.

Sharding is therefore an important technique for horizontal scaling.

It becomes useful when the complete dataset becomes too large for one database server.

Sharding and replication should not be confused.

In replication, multiple servers contain copies of the same data.

In sharding, multiple servers contain different portions of the data.

Easy way to remember:

Replication means copy.

Sharding means divide and distribute.

8. Distributed Databases

A distributed database is a database system in which data is stored across multiple machines or geographical locations, but these machines work together as part of the overall database system.

For example, a global company may have infrastructure in India, the United States and Europe.

The purpose is to provide better scalability, availability and performance for users across different geographical locations.

Distributed databases can use techniques such as replication and sharding.

Some information may be replicated across locations, while other information may be divided between different servers.

Distributed databases introduce additional challenges because machines must communicate over networks. Network failures, synchronization and consistency become important problems.

9. Data Centers and Geographical Distribution

A data center is a physical facility containing many servers, storage devices and networking systems.

Large internet companies can operate infrastructure across multiple geographical regions.

Suppose an Indian user wants to watch a popular video uploaded by a creator in the United States.

The user does not necessarily need to retrieve that video from a distant US server every time. If a cached copy of that content is available closer to the Indian user, the nearby infrastructure can serve it.

This reduces latency and improves loading speed.

This idea is commonly combined with caching and Content Delivery Networks.

10. Load Balancing in Databases

Load balancing means distributing incoming database requests across multiple available servers so that one server does not become overloaded.

Suppose an application receives one lakh read requests.

If all those requests are sent to one database server, that server may become slow.

If multiple replicas are available, the requests can be distributed among them.

A load balancer helps decide where incoming requests should be sent.

Load balancing provides:

- Better performance
- Better resource utilization
- Reduced server overload
- Improved availability
- Better scalability

Easy way to remember:

Load balancing means dividing the traffic, not dividing the data.

11. Caching in DBMS

Caching means storing frequently requested data in a faster temporary storage location so that the database does not need to perform the same work repeatedly.

Suppose an e-commerce website displays its top ten products on the homepage.

Millions of users may request the same information.

Instead of querying the main database every single time, the result can be stored in a cache.

Future users can receive the information directly from the cache.

Caching provides:

- Faster response time
- Reduced database load
- Lower latency
- Better application performance

Redis and Memcached are commonly used caching technologies.

One important problem is stale data.

Suppose the database contains a product price of ₹1,000, but the cache still contains the old price of ₹1,200.

The cache is now stale.

Therefore, applications need proper cache expiration and cache invalidation strategies.

Easy way to remember:

Caching means keeping frequently used data in faster temporary storage.

12. Read Replicas

A read replica is a replicated copy of a database that is primarily used to handle read operations.

Many applications have significantly more reads than writes.

For example, on a social media application, uploading a new post is a write operation.

But viewing profiles, viewing posts, reading comments and browsing feeds involve large numbers of read operations.

If all reads and writes go to one database server, it can become overloaded.

Instead, writes can go to the primary database while read requests are distributed among read replicas.

Read replicas therefore help with read scaling.

A read replica is not a completely different concept from replication. It is an application of database replication specifically designed to reduce read load.

13. High Availability

High availability means designing a database system so that it remains available even when some components fail.

Suppose an application has only one database server.

If that database server crashes, the entire application may stop working.

The database server is therefore a single point of failure.

To improve availability, we can maintain multiple database servers.

If the primary server fails, another server may take over its responsibility.

This process is called failover.

High availability systems commonly use:

- Replication
- Multiple servers
- Health monitoring
- Automatic failover
- Backups
- Load balancing

The objective is to minimize downtime.

Easy way to remember:

High availability means keeping the system available as much as possible.

14. Fault Tolerance

Fault tolerance means designing a system so that it can continue functioning even when one or more components fail.

Suppose a database system uses multiple servers.

If one server fails but the remaining servers can continue serving users, the system has fault-tolerant characteristics.

Fault tolerance usually requires redundancy. Instead of depending on a single component, important components have alternatives.

High availability and fault tolerance are closely related.

High availability focuses mainly on minimizing downtime.

Fault tolerance focuses on the system's ability to continue operating despite failures.

For example, if one database machine crashes and another immediately takes over, the system is designed to tolerate that failure.

15. CAP Theorem

CAP theorem is an important concept in distributed databases.

CAP represents three properties:

Consistency

Availability

Partition Tolerance

Consistency

Consistency means users should see a consistent view of the data.

Suppose an account balance changes from ₹10,000 to ₹8,000.

Ideally, when clients access different relevant nodes, they should not receive conflicting current values.

Consistency is particularly important in systems where incorrect or conflicting data can cause serious problems.

Availability

Availability means the system continues responding to valid requests through available nodes.

Even if some components experience problems, the system tries to remain available to users.

Partition Tolerance

Partition tolerance means the distributed system can continue operating even when a network problem prevents some servers from communicating with each other.

For example, suppose servers in India and the United States are running correctly, but a network failure temporarily prevents communication between them.

This situation is called a network partition.

Main Idea of CAP Theorem

In a distributed database, when a network partition occurs, the system generally has to make a trade-off between consistency and availability.

A system may prioritize consistency and refuse or delay certain operations rather than risk returning inconsistent data.

Another system may prioritize availability and continue responding, accepting that different nodes may temporarily have different versions of data.

The important interview point is:

CAP theorem is not simply about randomly choosing any two properties.

The important trade-off appears when a network partition occurs. At that time, the system must decide whether to prioritize consistency or availability.

16. Real-World Database Scaling Challenges

Database scaling solves many problems, but it also introduces new challenges.

Replication Lag

Changes made on the primary database may take some time to reach replicas.

Therefore, a user may occasionally read slightly older data from a replica.

Sharding Complexity

After data is divided across multiple servers, the application must determine which server contains the required data.

Queries involving information from several shards become more complicated.

Hotspots

Data may not always be distributed equally.

Suppose one shard receives 80% of the traffic while other shards receive very little traffic.

The heavily used shard becomes a hotspot.

Choosing a good sharding strategy is therefore very important.

Cache Invalidation

When information changes in the main database, the corresponding cached information must also be updated or removed.

Otherwise, users may receive outdated information.

Distributed Transactions

A single business operation may involve multiple databases.

For example, placing an order may involve an Orders database and a Payments database.

A difficult situation occurs when the payment succeeds but order creation fails.

Maintaining correctness across multiple databases is more complicated than working with a single database.

Network Failures

Distributed databases depend heavily on network communication.

Networks can fail or become slow.

This can create timeouts, delays and temporary inconsistencies between servers.

Failover Problems

When a primary database fails, the system must determine which replica should become the new primary.

This process must be handled carefully to avoid data loss or multiple servers incorrectly acting as primary.

Cost

Scaling also increases infrastructure cost.

More database servers mean additional hardware or cloud costs, monitoring, backups, networking and maintenance.

Quick Revision

Database scaling means increasing database capacity to handle growth.

Vertical scaling means making the same server more powerful.

Horizontal scaling means adding more servers.

Replication means maintaining the same data on multiple servers.

Primary-Replica architecture generally sends writes to the primary and can send reads to replicas.

Partitioning means dividing large data into smaller logical pieces.

Sharding means dividing data across multiple database servers.

A distributed database stores and manages data across multiple machines or locations.

Load balancing means distributing requests among servers.

Caching means keeping frequently accessed data in faster temporary storage.

Read replicas are database copies mainly used to handle read requests.

High availability means minimizing system downtime.

Fault tolerance means continuing operation even when components fail.

CAP theorem explains the trade-off between consistency and availability when a network partition occurs in a distributed system.

The easiest way to remember the major concepts is:

Replication = Copy the data.

Partitioning = Divide the data.

Sharding = Divide the data across servers.

Caching = Keep frequently used data in faster storage.

Load balancing = Divide the traffic.

Vertical scaling = Bigger server.

Horizontal scaling = More servers.